Transformer Mimarisi ve NLP'ye Giriş

Stanford CME295 · Transformers & LLMs · Autumn 2025 · Lecture 1 — Transformer

Berk KARATAŞ

Lisansüstü Ders — Ödev 2

Dizideki Yeri ve Önemi

- CME295'in amacı: Transformer'ların nasıl çalıştığını ve LLM'lerle ilişkisini öğrenmek; LLM'lerin nasıl eğitildiğini ve kullanıldığını görmek
- Ders 1, serinin temeli: sonraki dersler bu mimari üzerine kurulu
- Önceki ders yok (ilk ders). Ön bilgi: makine öğrenmesi temelleri ve lineer cebir
- Bu sunumun hedefi: metinden Transformer'a giden yolu adım adım anlamak

1	**Transformer**
2	Transformer tabanlı modeller ve püf noktaları
3	Büyük dil modelleri (LLM)
4	LLM eğitimi
5	LLM ayarlama (tuning)
6	LLM muhakemesi
7	Ajan tabanlı LLM'ler
8	LLM değerlendirme
9	Güncel trendler

Kaynak: Stanford CME295 Autumn 2025 müfredatı (cme295.stanford.edu/syllabus/2025)

Bu Derste Ne Öğreneceğiz?

1. NLP görevleri
2. Tokenization
3. Token temsili (Word2vec)
4. RNN ve LSTM
5. Attention
6. Transformer mimarisi
7. Uçtan uca örnek

Temel soru: Bilgisayar, cümledeki kelimelerin birbiriyle ilişkisini nasıl öğrenir? Cevap: Attention.

NLP Nedir ve Nereden Geldik?

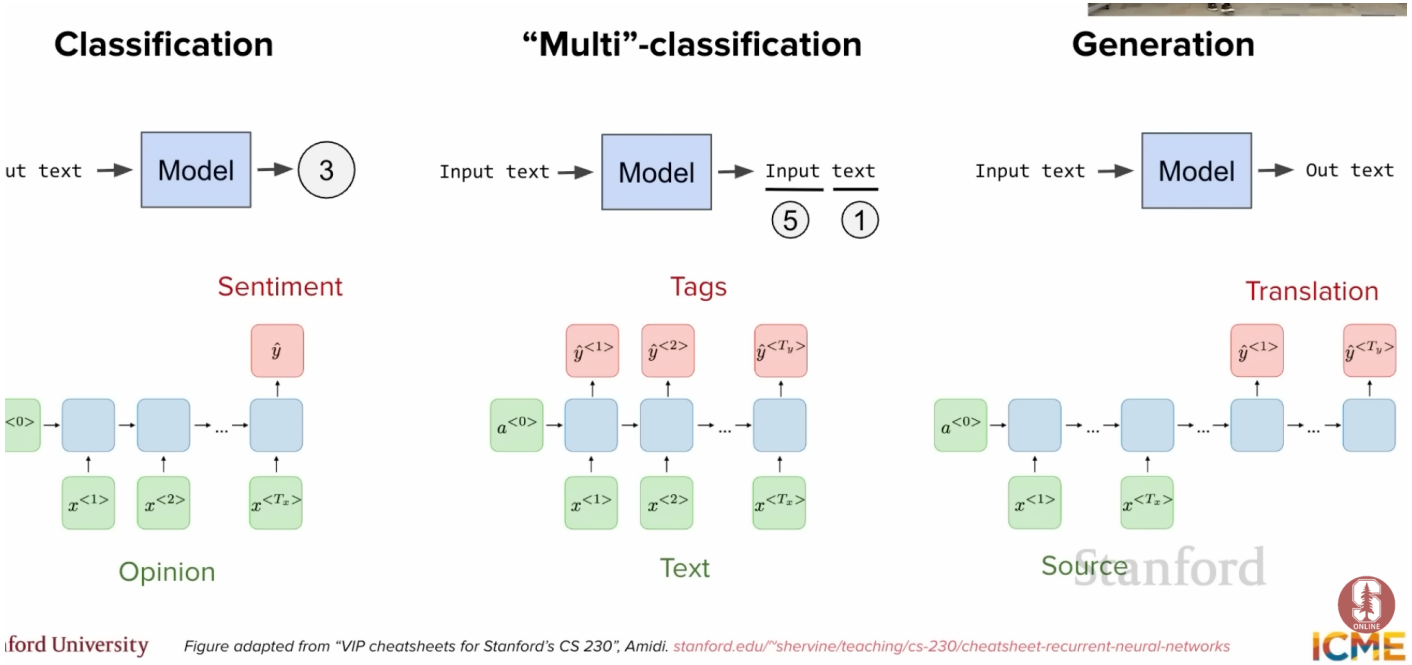
- NLP (Doğal Dil İşleme): bilgisayarların insan dilini anlaması, yorumlaması ve üretmesi
- Günlük hayatta: çeviri, sesli asistan, spam filtresi, sohbet botları
- Zorluk: dil bağlama, belirsizliğe ve kültürel kullanıma bağlıdır



Kuramsal temeller → çok veri ve artan hesaplama gücü → fikirlerin hızlı denenmesi

Kaynak: Stanford CME295, Ders 1 (zaman çizelgesi)

Temel NLP Görevleri



Görsel: Stanford CME295 Ders 1 (Amidi, CS 230 cheatsheet'inden uyarlanmış)

Hepsinde ortak fikir: model bir metin alır, görevine göre bir sayı, etiket dizisi ya da yeni metin üretir.

Sınıflandırma

Metin → tek çıktı. Duygu, niyet, dil tespiti, konu belirleme

"Çoklu" sınıflandırma

Her token için etiket. Sözcük türü (POS), varlık tanıma (NER)

Üretim

Metin → yeni metin. Çeviri, soru-cevap, özetleme

Üç Örnek Görev ve Metrikleri

Duygu analizi

Örnek: "Bu oyuncak ayı ÇOK SEVİMLİ!" → olumlu

Veri: Amazon yorumları, IMDB, Twitter

Metrikler: Accuracy, Precision, Recall, F1

Varlık tanıma (NER)

Metindeki her token için varlık etiketi üretilir

Veri: Reuters haber derlemi (CoNLL-2003)

Metrikler: aynı dörtlü; token düzeyinde, varlık türü başına

Makine çevirisi

Örnek: İngilizce → Fransızca

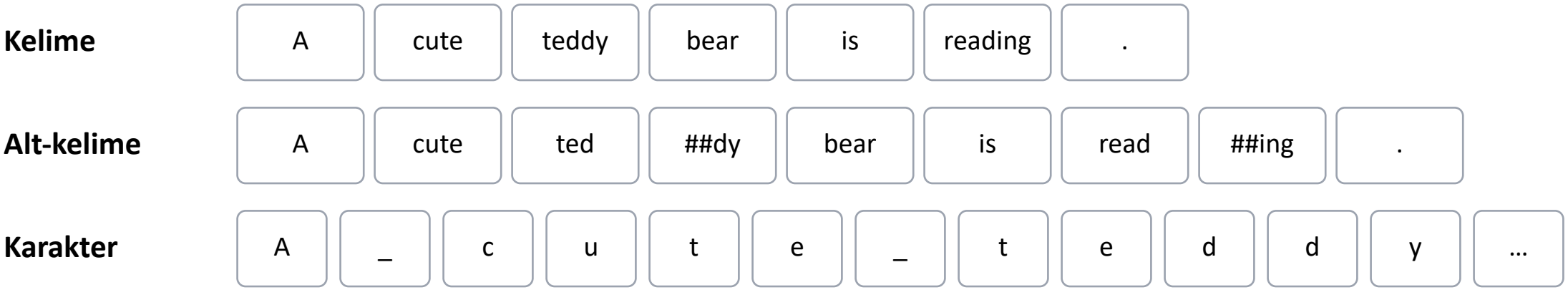
Veri: WMT'14

BLEU: çeviri kalitesi (precision benzeri)
ROUGE: üretim kalitesi (recall benzeri)
Perplexity: modelin kelimeleri görünce ne kadar "şaşırdığı"

Precision: pozitif dediklerimizin kaçı doğru? Recall: gerçek pozitiflerin kaçını yakaladık? F1: ikisini birleştiren skor

Tokenization: Metni Parçalara Ayırmak

Örnek cümle: "A cute teddy bear is reading."



Yöntem	Artıları	Eksileri
Kelime	Basit, yorumlanabilir	Sözlükte olmayan kelime (OOV) riski; kök bilgisini kullanmaz
Alt-kelime (WordPiece, BPE)	Ortak ön/son ekleri kullanır; veriden öğrenilir	OOV riski var ama kelimeye göre daha az
Karakter	OOV ihtimali çok düşük; yazım hatalarına dayanıklı	Hesaplama yavaşlar; embedding'ler yorumlanamaz

Kaynak: Stanford CME295, Ders 1

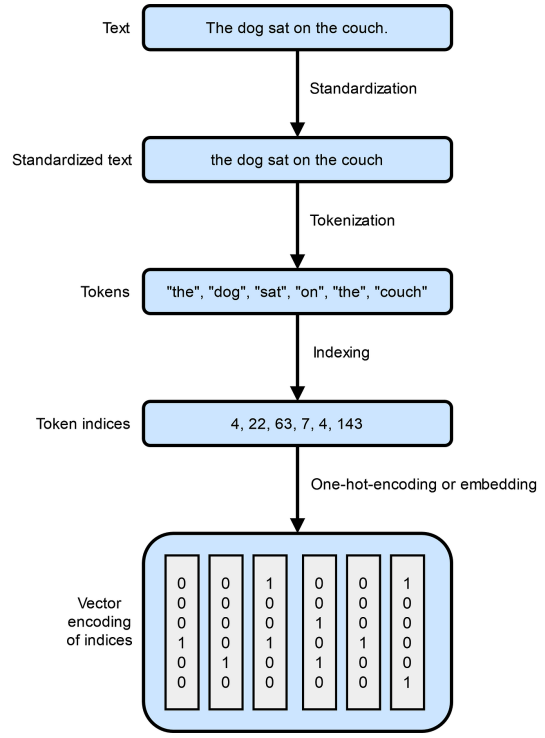
Token Temsili: One-Hot mu, Embedding mi?

One-hot kodlama

Her token, sözlük boyutu V uzunluğunda ve yalnızca bir elemanı 1 olan vektördür.
Sorun: vektörler çok büyük, kelimeler arası anlam ilişkisi yok.

Öğrenilmiş embedding

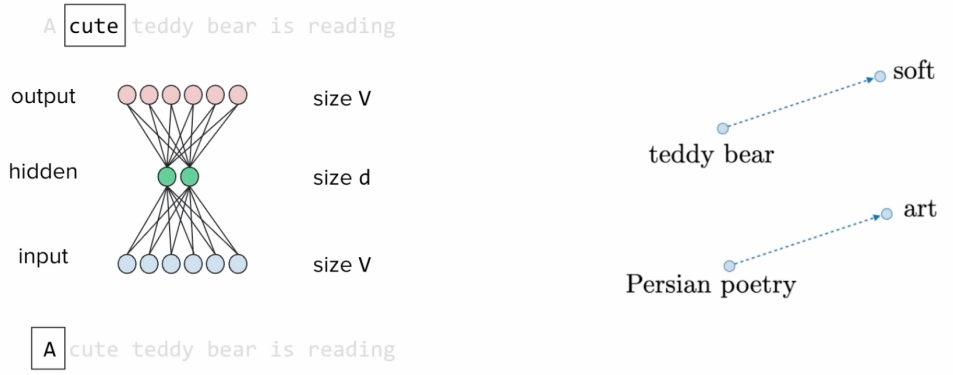
Her token, d boyutlu yoğun bir vektörle temsil edilir (d ≪ V).
Benzer anlamlı kelimeler vektör uzayında birbirine yakın olur.



Görsel: "The Art of Tokenization" (Medium, Data Science)

Word2vec: Kelimeleri Sayılara Çevirmek

- Amaç: kelimeleri bağlamlarından öğrenilen sayısal vektörlere (embedding) çevirmek
- Milyarlarca kelimelik metinde bir vekil görev (proxy task) çözen sinir ağı; gizli katman embedding'i verir
- CBOW: çevre kelimelerden ortadaki kelimeyi tahmin eder
- Skip-gram: ortadaki kelimeden çevresindekileri tahmin eder
- Mimari: giriş (V) → gizli (d) → çıkış (V)
- Sınır: kelime sırası sayılmaz; embedding bağlama duyarlı değildir

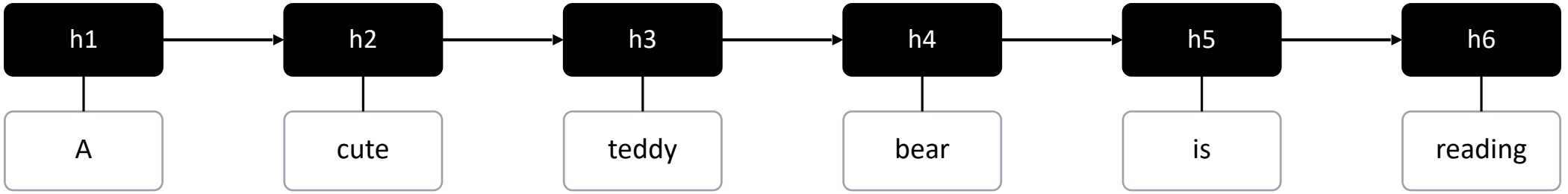


Görsel: Stanford CME295 Ders 1 · Mikolov et al., 2013

Not: Word2vec'teki "bag of words" ifadesi, frekans sayan klasik Bag of Words yönteminden farklıdır.

RNN: Cümleyi Sırayla Okumak

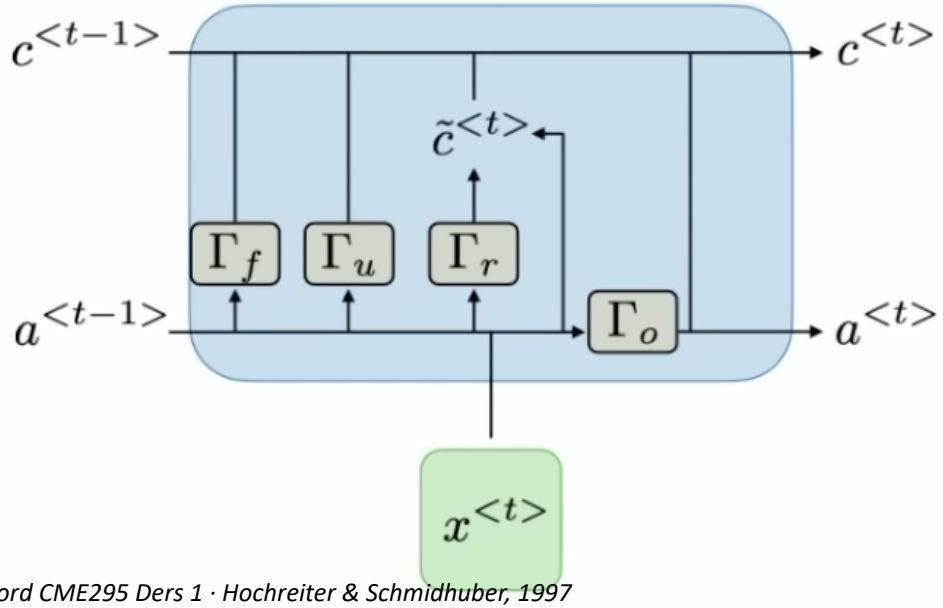
- 1980'lerde tanıtıldı; bağlantılar zaman içinde bir dizi oluşturur
- Her adımda yeni kelimeyi okur ve gizli durumu (hidden state) bir sonraki adıma aktarır
- Sınıflandırma, token etiketleme ve üretim için kullanılabilir
- Sorun 1: kaybolan gradyan — uzun bağımlılıklarda bilgi kaybolur
- Sorun 2: sıralı çalıştığı için yavaştır, eğitim paralelleştirilemez



Gizli durum h, her adımda bir sonrakine taşınır. Kaynak: Stanford CME295, Ders 1

LSTM: Kapılarla Daha Düzenli Bellek

- "Long Short-Term Memory" makalesiyle 1997'de tanıtıldı
- RNN'in hücre durumunu daha yapılandırılmış yönetir
- Kapılar (Γ) hangi bilginin tutulup unutulacağını ve çıktıya verileceğini kontrol eder
- Uzun bağımlılıkları RNN'e göre daha iyi korur
- Ama hâlâ sıralı çalışır; paralelleştirme sınırlıdır



Görsel: Stanford CME295 Ders 1 · Hochreiter & Schmidhuber, 1997

Özet: Word2vec ve RNN/LSTM

Yöntem	Artıları	Eksileri
Word2vec (CBOW, Skip-gram)	Çok basit ama güçlü; sezgisel embedding'ler	Kelime sırası sayılmaz; embedding bağlama duyarlı değil
RNN (RNN, LSTM)	Kelime sırası önemlidir; güçlü sonuçlar	Kaybolan gradyan; yavaş hesaplama

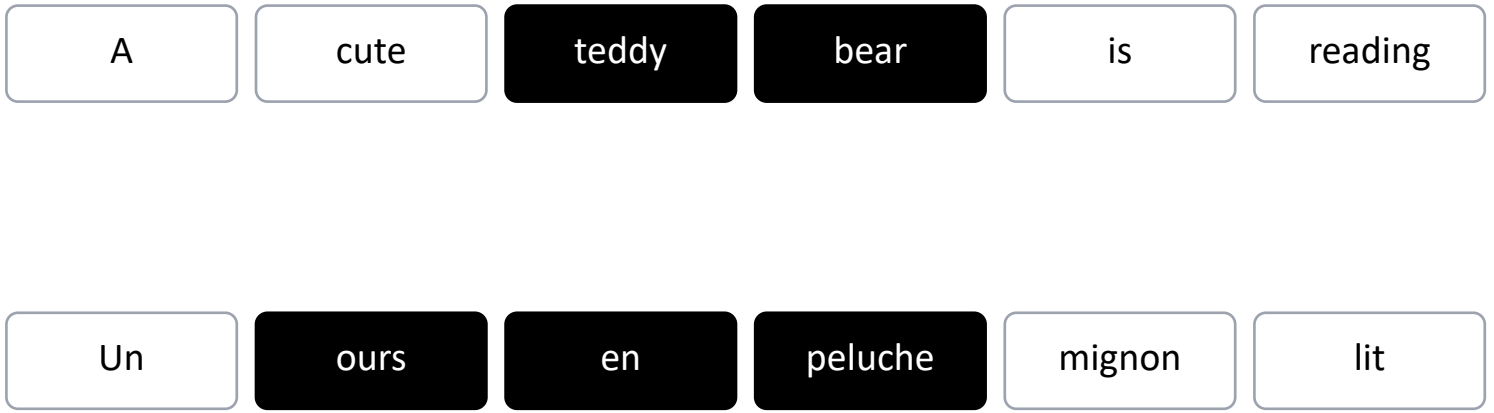
Eksik olan: hem kelime sırasını bilen hem uzun mesafeli ilişkileri yakalayan hem de paralel çalışabilen bir yöntem. Çözüm: attention.

Kaynak: Stanford CME295, Ders 1

Attention'ın Doğuşu (2014)

- Çeviri görevlerinde uzun mesafeli bağımlılıklar gerçek bir sorundu
- Seq2seq model, girdi cümlesinin ne dediğini "hatırlayamıyordu"
- Bahdanau ve arkadaşları (2014): çıktı üretirken girdinin ilgili kısmına bakmayı önerdi

Örnek (derste de kullanılan cümle):



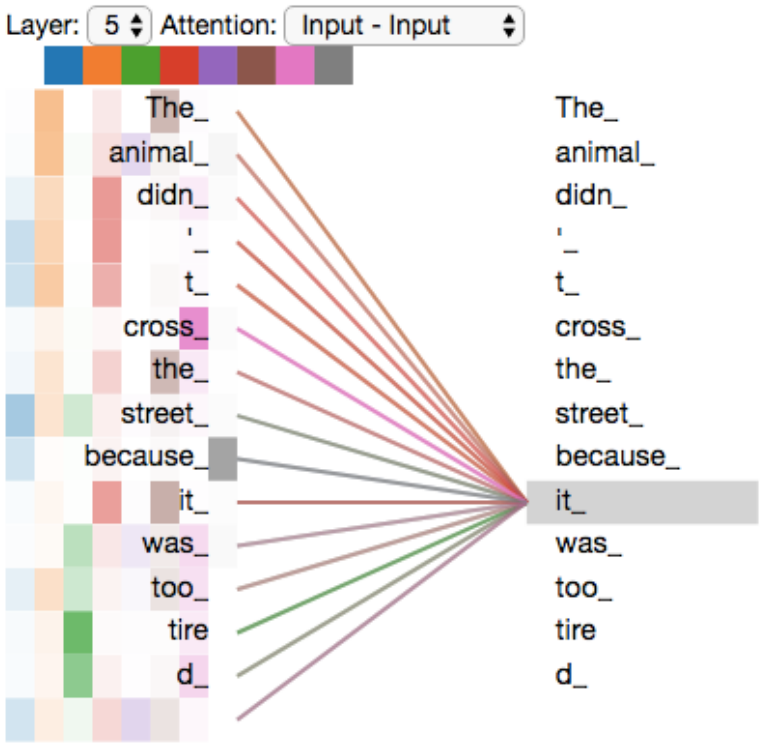
"ours en peluche" üretilirken model "teddy bear"a odaklanır

Kaynak: Bahdanau et al., 2014 · Stanford CME295, Ders 1

Self-Attention: Query, Key, Value

- Query (Q): "Ne arıyorum?" — Key (K): "Hangi bilgi var?" — Value (V): "Gerçek içerik"
- Kütüphane benzetmesi: Q aradığın konu, K kitap etiketleri, V kitabın içeriği
- Her token'ın Q'su, tüm K'larla karşılaştırılır; skorlar softmax ile ağırlığa dönüşür
- Çıktı: V'lerin bu ağırlıklarla alınmış ağırlıklı ortalaması
- Hepsi matris çarpımlarıyla aynı anda hesaplanır → paralel

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q \cdot K^T / \sqrt{d_k} \right) \cdot V$$



Görsel: Alammar (2018), The Illustrated Transformer. "it" kelimesinin diğer kelimelerle ilişkisi

Küçük Örnek: "Kedi süt içti"

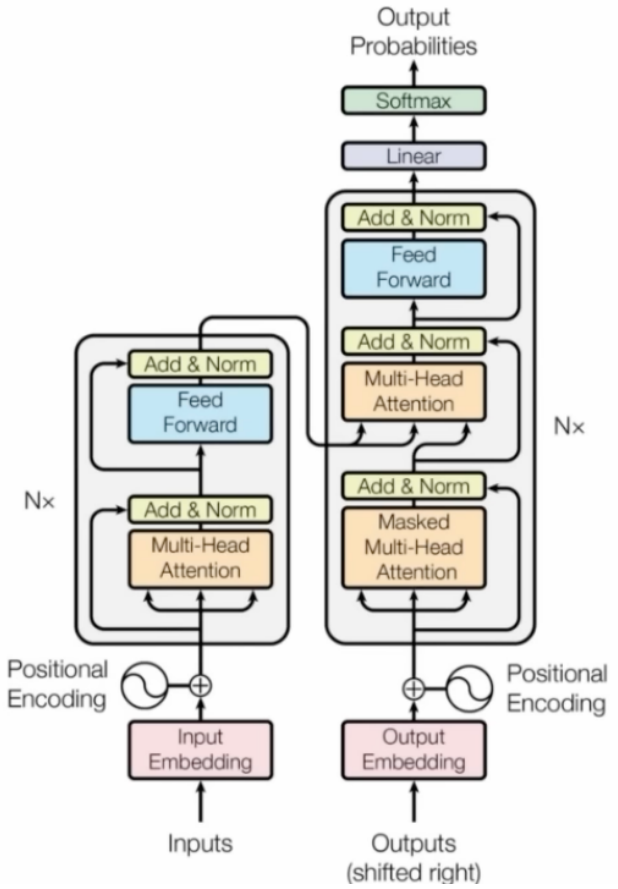
"içti" kelimesi için Query q = [2, 0], d_k = 2. (Sayılar öğretici amaçla seçilmiştir.)

Token	Key (K)	Value (V)	Skor = q·K/√2	Softmax ağırlığı
kedi	[2, 0]	[1, 0]	2.83	0.77
süt	[1, 1]	[0, 1]	1.41	0.19
içti	[0, 1]	[1, 1]	0.00	0.05

Çıktı = 0.77·[1,0] + 0.19·[0,1] + 0.05·[1,1] ≈ [0.81, 0.23]
"içti", en çok "kedi"ye (ağırlık %77) bakarak kendi yeni temsilini oluşturdu.

Gerçek modelde Q, K, V; embedding'lerin öğrenilen Wq, Wk, Wv matrisleriyle çarpılmasıyla üretilir.

Transformer'a Genel Bakış



- 2017, "Attention Is All You Need" makalesi
- Yalnızca self-attention mekanizmasına dayanır
- Çeviri görevlerinde o dönemin en iyi sonuçlarını verdi

MHA

Multi-head attention: self-attention ve encoder-decoder attention

FFN

Her pozisyonda ayrı dönüşüm uygular

Positional Enc.

Token sırası bilgisini ekler

Görsel: Vaswani et al. (2017), Stanford CME295 Ders 1'den uyarlanmış

Girdi ve Konum Bilgisi

- Metin önce token'lara ayrılır
- Her token için öğrenilmiş bir embedding alınır (V: sözlük boyutu, d_model: embedding boyutu)
- Hile: Attention sırayı bilmez, bu yüzden girdiye konum bilgisi eklenir
- Konum kodlaması öğrenilmiş de olabilir, sabit (hardcoded) de
- Amaç: modelin tokenlar arası göreli konumu anlaması

Konum farkındalıklı embedding = token embedding + konum bilgisi

Encoder, Decoder ve Çıkış

Encoder

Girdiyi işler.

- Self-attention
- Feed-forward ağı
- Normalizasyon (Add & Norm)
- N kat üst üste

Çıktı: bağlamı bilen (context-aware) embedding'ler

Decoder

Çıktıyı adım adım üretir.

- Çeviride [BOS] ile başlar
- Maskeli self-attention
- Encoder-decoder attention
- Feed-forward + normalizasyon
- N kat

Çıkış

- Linear katman + Softmax
- Sınıflandırma problemi: her sınıf bir kelime
- Her kelime için olasılık üretir
- [EOS] üretilince durur

Kaynak: Vaswani et al. (2017) · Stanford CME295, Ders 1

Hesaplama Püf Noktaları

Multi-Head Attention

- Birden fazla self-attention katmanı paralel çalışır
- Her başlık farklı ilişki türlerini yakalayabilir
- Çıktılar birleştirilir
- Benzetme: görüntü işlemede bir katmandaki çoklu CNN filtreleri

Label Smoothing

- 2015 bir görüntü makalesinden: aşırı özgüven kötüdür
- Gerçek etiketlere gürültü eklenir
- Aşırı öğrenmeyi (overfitting) azaltır; doğruluk ve BLEU'yu iyileştirir
- Transformer katmanı değil, genel bir eğitim tekniğidir

Kaynak: Stanford CME295, Ders 1

Uçtan Uca Örnek: Çeviri

"A cute teddy bear is reading." → "Un ours en peluche mignon lit."

1. Giriş

Token'lara ayır; [BOS] … [EOS] ekle

2. Embedding

Token embedding + konum bilgisi

3. Encoder

Wq, Wk, Wv → Q, K, V → h başlık self-attention → FFN

4. Kodlama

Bağlamı bilen embedding'ler decoder'a gider

5. Decoder

[BOS] ile başlar; self-attn + encoder-decoder attn + FFN + softmax

6. Üretim

Un → ours en peluche → mignon → lit → [EOS]

Kaynak: Stanford CME295, Ders 1 (uçtan uca örnek)

Üç Temel Çıkarım

1. Model metinle değil sayılarla çalışır: metin → token → embedding (+ konum bilgisi).

2. RNN/LSTM, kaybolan gradyan ve sıralı işlem yüzünden zorlanır; attention bu iki sorunu azaltır.

3. Attention ve Transformer, modern LLM'lerin temel yapı taşıdır.

Kavrama Soruları

1 One-hot kodlamanın en büyük sınırlılığı nedir? Word2vec bunu nasıl çözer?

2 RNN'lerin uzun cümlelerde zorlanmasının temel nedeni nedir?

3 Self-attention, RNN'e göre hangi avantajları sağlar?

Sonraki Derse Geçiş

Transformer'ı öğrendik. Peki bu mimari nasıl hızlandırılır, uzun metinlerle nasıl çalışır ve hangi model aileleri vardır?

Ders 2: Transformer tabanlı modeller ve püf noktaları

- Attention yaklaşıklaması; MHA, MQA, GQA
- Konum embedding'leri (klasik, öğrenilmiş) ve RoPE
- Transformer tabanlı mimariler
- BERT ve türevleri

Kaynak: cme295.stanford.edu/syllabus/2025

Kaynaklar

- Stanford CME295, Autumn 2025, Lecture 1 — Transformer (video ve resmî slaytlar): cme295.stanford.edu
- Vaswani et al., "Attention Is All You Need", 2017
- Bahdanau et al., "Neural Machine Translation by Jointly Learning to Align and Translate", 2014
- Mikolov et al., "Efficient Estimation of Word Representations in Vector Space", 2013
- Hochreiter & Schmidhuber, "Long Short-Term Memory", 1997
- Alammar, J. (2018). The Illustrated Transformer: jalammar.github.io/illustrated-transformer
- Amidi, A. & S. (2024). Super Study Guide: Transformers & Large Language Models
- "The Art of Tokenization: Breaking Down Text for AI", Medium, Data Science